

Boas Práticas Git em Computador Compartilhado

Boas Práticas — Git em Computador Compartilhado

0. Contexto: por que isso é diferente do seu PC pessoal

No seu computador pessoal, usar `--global` e deixar chave SSH salva é normal e seguro, porque só você tem acesso. Numa máquina **compartilhada** (empresa, laboratório da faculdade, computador de terceiros), qualquer configuração ou credencial que você deixa salva pode ser usada por **outra pessoa em seu nome**. O cuidado aqui é 100% sobre isolar seu acesso e não deixar rastros de autenticação.

1. Quando entrar em um computador novo (antes de qualquer coisa)

Antes de configurar nada, verifique o que já existe ali — pode ser que outra pessoa tenha esquecido configurações ou credenciais:

```
git config --list --global
```

Mostra se já existe alguma identidade configurada globalmente na máquina (de outra pessoa).

```
ls ~/.ssh
```

Verifica se já existem chaves SSH salvas ali (de outra pessoa, possivelmente esquecidas).

Se encontrar algo de outra pessoa: não mexa, não apague — apenas não utilize, e siga com sua própria configuração isolada (Seção 2). Avise a pessoa responsável pela máquina se parecer um esquecimento sério (chave SSH ativa, por exemplo).

2. Configurando esse PC (do jeito seguro, sem afetar outras pessoas)

Identidade — SEM usar `--global`

```
cd pasta-do-projeto
git config user.name "Seu Nome"
git config user.email "seuemail@exemplo.com"
```

Sem a flag `--global`, essa configuração vale **só para esse repositório específico**, ficando salva dentro de `.git/config` daquela pasta. Não interfere em nada fora dali.

Conferindo que ficou local (não global)

```
git config --list --local
```

Autenticação — prefira HTTPS + Token temporário em vez de SSH

Numa máquina compartilhada, evite criar chave SSH permanente (a chave privada fica salva no disco). Prefira:

```
git clone https://github.com/usuario/repositorio.git
```

Quando pedir usuário/senha, use seu usuário do GitHub + um **token temporário** (não sua senha de verdade — o GitHub não aceita mais senha comum).

Como gerar o token temporário

1. GitHub → foto de perfil → **Settings**
2. **Developer settings** → **Personal access tokens** → **Tokens (classic)**
3. **Generate new token**
4. Defina uma **expiração curta** (ex: 1 dia, ou o tempo que for usar a máquina)
5. Marque só a permissão `repo`
6. Copie o token (ele só aparece uma vez — cole em algum lugar temporário até usar)

3. Trabalhando normalmente (fluxo do dia a dia, igual sempre)

```
git status
git add .
git commit -m "mensagem"
git pull
git push
```

Nada muda aqui — a diferença toda está na configuração (Seção 2) e na limpeza (Seção 4).

4. Apagando/limpando ao sair (checklist obrigatório)

1. Remova a configuração local do projeto

```
git config --unset user.name
git config --unset user.email
```

2. Se por algum motivo criou chave SSH, apague-a do computador

```
rm ~/.ssh/id_ed25519
rm ~/.ssh/id_ed25519.pub
```

E revogue ela no GitHub também (isso é o passo que realmente importa): GitHub → Settings → **SSH and GPG keys** → encontre a chave → **Delete**

(apagar só do computador não invalida a chave — se alguém copiou antes, ela continua funcionando até você revogar no GitHub)

3. Revogue o token de acesso (se usou HTTPS + PAT)

GitHub → Settings → **Developer settings** → **Personal access tokens** → encontre o token → **Delete**

Isso invalida o token na hora, mesmo que tenha ficado salvo em algum lugar da máquina.

4. Limpe credenciais salvas automaticamente pelo Windows

```
git credential-manager github logout
```

Ou manualmente: **Painel de Controle** → **Contas de Usuário** → **Gerenciador de Credenciais** → remova qualquer entrada com `github.com`.

5. Apague a pasta do projeto clonado

```
rm -rf pasta-do-projeto
```

Isso remove código, histórico local, e qualquer configuração `--local` esquecida.

5. Checklist resumido (para revisar rapidinho antes de sair)

Etapa	Ação
☐	<code>git config --unset user.name</code> e <code>user.email</code>
☐	Apagar chave SSH do disco (se criou)
☐	Revogar chave SSH no GitHub (se criou)
☐	Revogar token de acesso no GitHub (se usou)
☐	Limpar Gerenciador de Credenciais do Windows
☐	Apagar a pasta do projeto clonado

6. Por que isso importa de verdade

Deixar uma chave SSH ou token válido esquecido numa máquina compartilhada é um dos vetores mais comuns de vazamento de acesso em empresas reais — alguém pode enviar código em seu nome, acessar repositórios privados, ou pior, comprometer um projeto inteiro sem que ninguém perceba de onde veio. Ter esse cuidado desde a faculdade já é uma prática profissional que faz diferença.

7. Regra de ouro para lembrar sempre

Em máquina compartilhada: configure local, autentique com token temporário, e limpe tudo antes de sair. Em PC pessoal: `--global` e SSH permanente continuam sendo a forma mais prática e correta.